

Modelado Semántico de la Web

# TurismoSemántico

Plataforma inteligente del patrimonio cultural de España

---

Memoria del trabajo final

Integración de datos abiertos, RDF/OWL, SPARQL y búsqueda semántica

---

**Autoría:** Sergio Alves Martín

**Repositorio:** ProyectoFinal\_MSW\_TurismoSemantico

**Fecha:** Mayo de 2026

# Índice

|                                           |   |
|-------------------------------------------|---|
| Resumen ejecutivo y trazabilidad          | 1 |
| 1. Descripción del dominio elegido        | 1 |
| 2. Fuentes de datos abiertas utilizadas   | 2 |
| 3. Limpieza, integración y preprocesado   | 2 |
| 4. Tecnologías propuestas y justificación | 3 |
| 5. Arquitectura del sistema               | 3 |
| 6. Funcionalidades de la aplicación       | 4 |
| 7. Diseño de la interfaz de usuario       | 4 |
| 8. Evaluación y demostración              | 5 |
| 9. Valoración técnica y conclusiones      | 6 |

## Resumen ejecutivo y trazabilidad

La aplicación desarrollada no se queda en un diseño teórico: implementa una plataforma web funcional que integra datos abiertos, construye un grafo RDF/OWL, permite consultas SPARQL y añade recuperación semántica con embeddings y chatbot RAG. La siguiente tabla resume la correspondencia directa con la guía del trabajo.

| Indicación de la guía        | Respuesta en TurismoSemántico                                                                                                    |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Dominio elegido              | Turismo cultural y patrimonio español, con destinos, museos, sitios UNESCO, POIs históricos y clima.                             |
| Dos fuentes abiertas         | Wikidata, OpenStreetMap/Nominatim/Overpass y Open-Meteo; Anthropic es opcional y no sustituye a las fuentes abiertas.            |
| Limpieza y preprocesado      | Normalización de coordenadas, tipos y textos; deduplicación por identificador/nombre; caché JSON; serialización TTL; embeddings. |
| Arquitectura tecnológica     | Frontend web, backend Flask, módulos de ingesta, RDFLib, ChromaDB, APIs externas y persistencia local.                           |
| Cuatro funcionalidades o más | Mapa, POIs por ciudad, búsqueda semántica, chatbot RAG, grafo RDF, SPARQL y descarga Turtle.                                     |
| Interfaz y flujo             | Pantalla de carga, navegación por secciones, mapa con panel lateral, buscador, chat, grafo y consola SPARQL.                     |
| Elementos valorables         | Recomendación semántica, chatbot, accesibilidad, SHACL, integración REST y exportación de conocimiento.                          |

## 1. Descripción del dominio elegido

TurismoSemántico es una aplicación web orientada al **turismo cultural y patrimonial en España**. El dominio elegido combina información geográfica, descripciones culturales, bienes declarados Patrimonio de la Humanidad, museos, monumentos y datos meteorológicos de apoyo a la visita. La motivación principal es que la información turística pública existe, pero está repartida entre fuentes heterogéneas: Wikidata publica entidades semánticas consultables con SPARQL, OpenStreetMap contiene puntos de interés geoespaciales mantenidos por la comunidad y Open-Meteo ofrece meteorología en tiempo real.

El programa transforma esas fuentes en una experiencia única: un mapa interactivo, un grafo RDF/OWL descargable, una consola SPARQL, un buscador por significado y un guía conversacional con recuperación aumentada por generación (RAG). El uso de modelado semántico está justificado porque los destinos no se tratan solo como filas de una tabla, sino como entidades enlazables con tipos, propiedades, identificadores externos y relaciones reutilizables.

## 2. Fuentes de datos abiertas utilizadas

| Fuente                             | Formato              | Datos obtenidos                                                                                                               | Uso en la app                                                          |
|------------------------------------|----------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Wikidata Query Service             | SPARQL/<br>JSON      | Sitios UNESCO de España, museos, ciudades y destinos con coordenadas, etiquetas, descripciones, imágenes e identificadores Q. | Fuente semántica principal, grafo RDF, embeddings y consultas.         |
| OpenStreetMap Overpass y Nominatim | JSON/<br>Overpass QL | Monumentos, castillos, ruinas, lugares de culto, museos y POIs por ciudad o por caja geográfica nacional.                     | Enriquecimiento geográfico, búsqueda de POIs y carga en segundo plano. |
| Open-Meteo                         | JSON REST            | Tiempo actual, viento y previsión de varios días a partir de latitud y longitud.                                              | Panel de detalle del destino.                                          |
| Anthropic API (opcional)           | JSON REST            | Generación de respuestas en lenguaje natural a partir del contexto recuperado.                                                | Chatbot RAG; si no hay clave, se usa respuesta local.                  |

En la caché local analizada (`data/destinos.json`) hay **105 destinos**: 54 ciudades o destinos generales, 30 museos y 21 elementos UNESCO, todos con identificador de Wikidata. El pipeline implementado permite completar esos datos con OpenStreetMap durante una ejecución con acceso a red, ya sea en segundo plano mediante `get_patrimonio_nacional` o de forma interactiva por ciudad mediante `/api/overpass`.

La selección de fuentes busca complementar tres dimensiones: **semántica** (Wikidata aporta identificadores, tipos y descripciones enlazables), **geoespacial** (OSM aporta puntos de interés actualizados por la comunidad) y **contextual** (Open-Meteo añade utilidad inmediata para planificar visitas). Esta combinación hace que los datos sean reutilizables, explicables y adecuados para funciones de exploración y recomendación.

## 3. Limpieza, integración y preprocesado

El proceso de datos está orquestado desde `app.py` y se divide en cinco fases:

1. **Extracción**: se ejecutan consultas SPARQL contra Wikidata para patrimonio UNESCO, museos y destinos generales. En OSM se usa Nominatim para geocodificar ciudades y Overpass QL para POIs turísticos. Open-Meteo se consulta bajo demanda con coordenadas.
2. **Normalización**: las coordenadas de Wikidata se convierten desde `Point(lon lat)` a campos numéricos `lat` y `lon`; los tipos se homogeneizan en `tipo` o `tipo_osm`; los textos de búsqueda se normalizan eliminando acentos, pasando a minúsculas y reduciendo caracteres no alfanuméricos.
3. **Deduplicación**: la función `_merge_destinos` prioriza claves estables: `wikidata_id`, `id` y, como último recurso, el nombre normalizado. Así evita duplicados entre consultas de Wikidata y entre Wikidata/OSM.
4. **Persistencia y caché**: los destinos se guardan en `data/destinos.json`; el grafo se serializa en `data/grafos.ttl`; los vectores se almacenan en `chroma_db/`. Si la caché existe, la aplicación puede arrancar sin repetir todas las consultas externas.
5. **Indexación semántica**: se construye un texto indexable con nombre, tipo, ciudad y descripción. Se generan embeddings con `paraphrase-multilingual-MiniLM-L12-v2`; si el entorno no dispone de `sentence-transformers`, se activa un fallback TF-IDF/Jaccard.

El código añade resiliencia práctica: reintentos y timeouts en APIs externas, varios endpoints Overpass alternativos, conversión de caché antigua a UTF-8, carga OSM en hilo de fondo para no bloquear la interfaz, y degradación controlada cuando faltan spaCy, ChromaDB o una clave LLM.

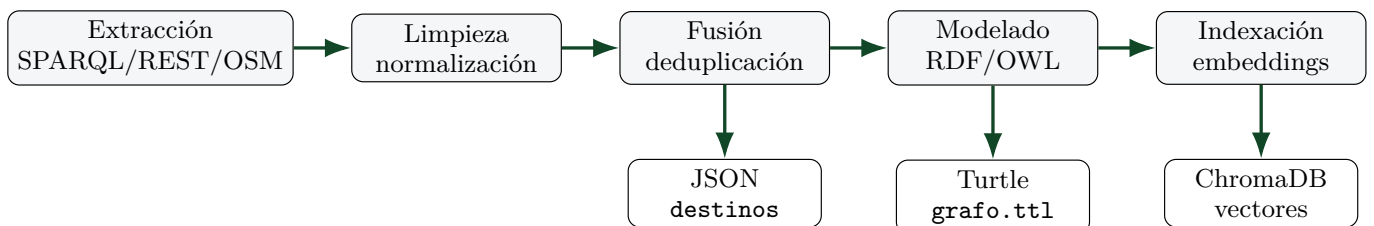


Figura 1: Pipeline de integración, preprocesado y persistencia.

## 4. Tecnologías propuestas y justificación

| Capa        | Tecnología                              | Justificación                                                                                                      |
|-------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Frontend    | HTML5, CSS3, JavaScript, Leaflet, D3.js | Permiten una interfaz web ligera con mapa, navegación por secciones, visualización de grafos y comunicación REST.  |
| Backend     | Flask y <b>requests</b>                 | Flask es suficiente para una API académica clara; <b>requests</b> simplifica el consumo de SPARQL/REST/Overpass.   |
| Semántica   | RDFLib, RDF/OWL, Turtle, SPARQL local   | RDFLib permite construir y consultar un grafo propio; Turtle facilita la entrega y reutilización del conocimiento. |
| Validación  | SHACL                                   | Se definen shapes para exigir nombre, coordenadas y año en patrimonio UNESCO.                                      |
| Búsqueda IA | SentenceTransformers, ChromaDB, spaCy   | Añaden búsqueda por similitud, persistencia vectorial y extracción de entidades en español.                        |
| Chatbot     | Pipeline RAG y Anthropic opcional       | El asistente responde usando destinos recuperados; sin API key mantiene respuesta local explicable.                |
| Datos       | JSON, TTL y caché local                 | JSON agiliza la carga de la app; TTL representa el modelo semántico interoperable.                                 |

## 5. Arquitectura del sistema

La arquitectura es modular. El navegador consume una API REST Flask; el backend coordina fuentes externas, caché, grafo RDF y búsqueda vectorial. Los módulos de la carpeta `pipeline/` separan responsabilidades: `wikidata.py`, `overpass.py`, `weather.py`, `rdf_model.py` y `embeddings.py`.

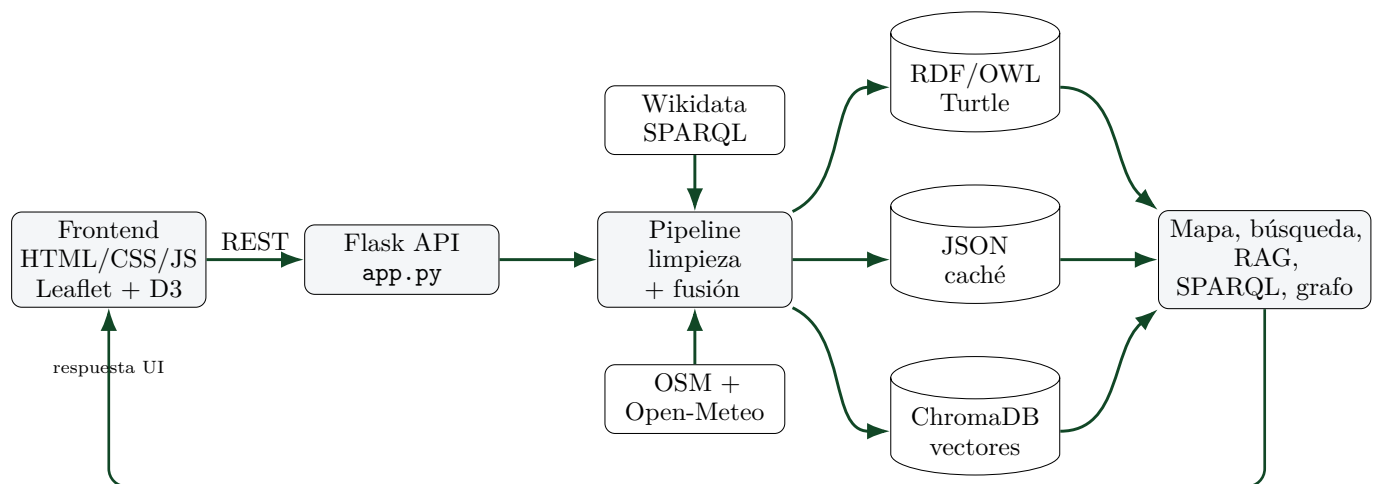


Figura 2: Arquitectura funcional de TurismoSemántico.

La API expone los endpoints principales: `/api/cargar`, `/api/estado`, `/api/destinos`, `/api/buscar`, `/api/chat`, `/api/grafos`, `/api/sparql`, `/api/shacl`, `/api/ttl`, `/api/clima` y `/api/overpass`. Esta separación permite que cada pantalla consuma solo los datos necesarios.

### Modelo semántico

La ontología propia usa el espacio de nombres `http://turismo-semantico.es/ontologia#`. Define las clases `Destino`, `PatrimonioUNESCO`, `Museo`, `Castillo`, `Ciudad` y `Evento`. `PatrimonioUNESCO`, `Museo` y `Castillo` son subclases de `Destino`. Las propiedades principales son `nombre`, `descripcion`, `latitud`, `longitud`, `urlImagen`, `wikidataId`, `anioPatrimonio`, `estaEn`, `tienePatrimonio` y `tieneEvento`. Cuando existe `wikidata_id`, el recurso local se enlaza con Wikidata mediante `owl:sameAs`.

El TTL local analizado contiene 105 bloques de destino, 105 individuos nombrados y 105 enlaces `owl:sameAs`. Las shapes SHACL incluidas validan que todo destino tenga nombre, latitud y longitud, y que un elemento UNESCO tenga año de declaración cuando se declara como `PatrimonioUNESCO`.

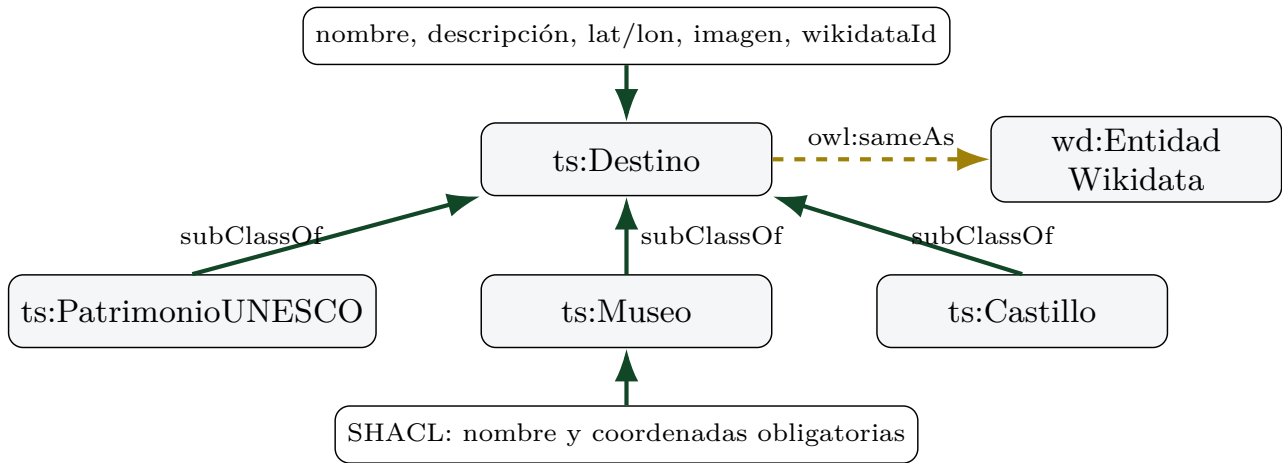


Figura 3: Esquema de la ontología y validaciones principales.

## 6. Funcionalidades de la aplicación

| Funcionalidad      | Objetivo y datos utilizados                                                 | Interacción del usuario                                                        |
|--------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Inicialización     | Cargar APIs o caché; informar de destinos, tripletas, OSM y embeddings.     | Botones de carga, log vivo y badge de estado.                                  |
| Mapa cultural      | Explorar destinos Wikidata/OSM con coordenadas, tipo, imagen y descripción. | Filtro por tipo, marcadores, popup y panel de detalle con clima.               |
| POIs por ciudad    | Encontrar monumentos, museos y castillos cercanos con Nominatim/Overpass.   | Introducir ciudad; la app añade resultados al mapa.                            |
| Búsqueda semántica | Recomendar destinos por significado usando embeddings, ChromaDB y NER.      | Consulta libre, ejemplos guiados, tarjetas con similitud y navegación al mapa. |
| Guía RAG           | Responder preguntas con contexto recuperado de los destinos más relevantes. | Chat conversacional con fuentes usadas; LLM opcional o respuesta local.        |
| Grafo RDF/OWL      | Mostrar el conocimiento generado y las relaciones semánticas.               | Visualización D3, tabla de tripletas y descarga <b>grafo.ttl</b> .             |
| SPARQL local       | Permitir explotación directa del grafo con consultas reproducibles.         | Editor con plantillas: destinos, UNESCO, museos y conteo por tipo.             |

## 7. Diseño de la interfaz de usuario

La interfaz está organizada como una aplicación de una sola página. Tras la pantalla de carga, el usuario navega con una cabecera fija entre cinco vistas: Explorar, Buscar, Guía IA, Grafo RDF y SPARQL. El flujo de interacción principal es:

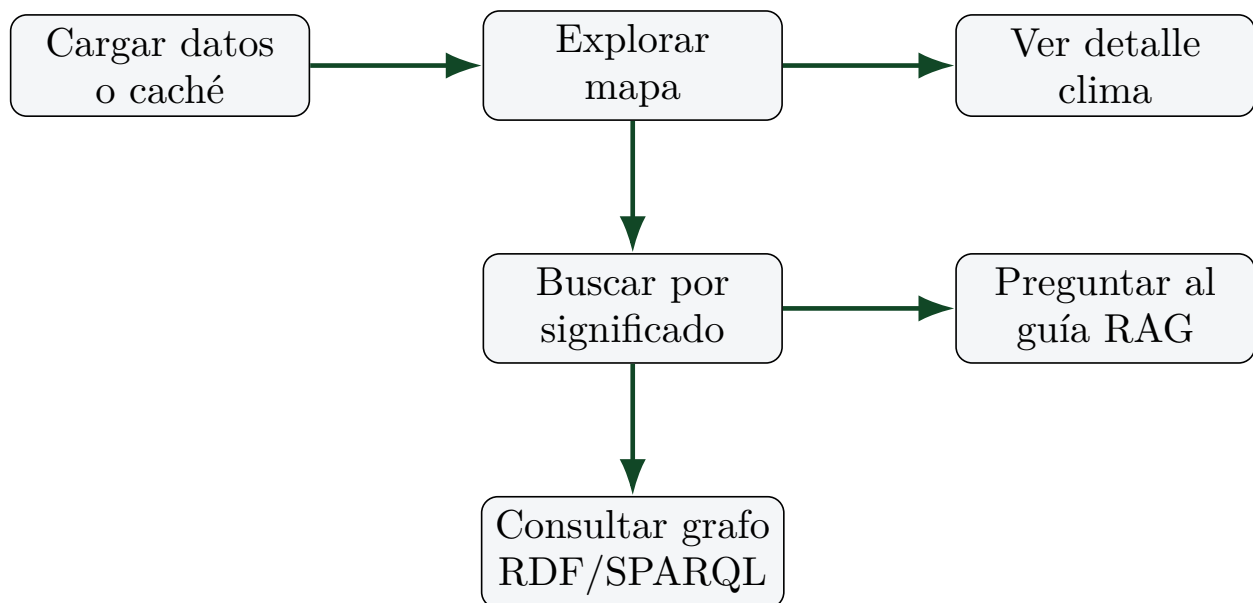


Figura 4: Flujo de uso principal de la interfaz.

El diseño prioriza claridad operativa: filtros en un panel lateral, tarjetas de resultados, indicadores de similitud, mensajes de carga, ejemplos predefinidos para búsqueda y SPARQL, y actualización periódica del estado cuando OSM termina en segundo plano. Además, incorpora criterios de accesibilidad: uso de `aria-label`, regiones `aria-live`, navegación por botones, foco visible, contraste alto, modo claro/oscuro, adaptación responsive y respeto de `prefers-reduced-motion`.

| Pantalla     | Componentes principales                             | Criterios de usabilidad/accesibilidad                                      |
|--------------|-----------------------------------------------------|----------------------------------------------------------------------------|
| Carga        | Fuentes, botones, log y estado.                     | Feedback continuo, carga en segundo plano y mensajes de error.             |
| Explorar     | Panel lateral, filtros, mapa Leaflet y detalle.     | Información progresiva: vista global primero, detalle solo al seleccionar. |
| Buscar/Chat  | Entrada libre, ejemplos, resultados y conversación. | Reduce fricción con ejemplos y conserva trazabilidad de fuentes.           |
| Grafo/SPARQL | D3, tabla de tripletas, editor y resultados.        | Aprendizaje guiado mediante plantillas y descarga interoperable.           |

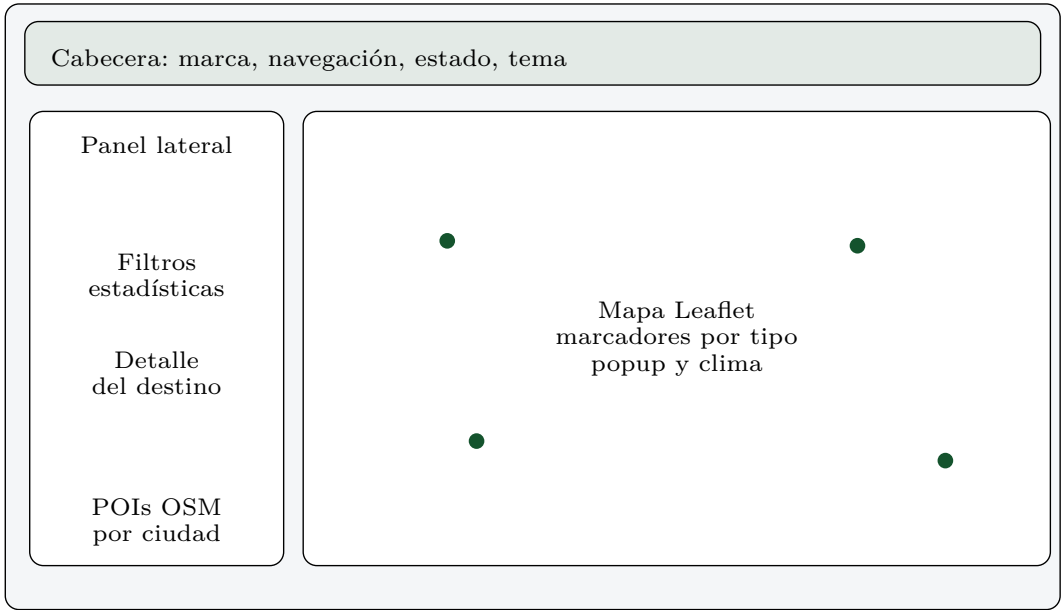


Figura 5: Boceto de la pantalla principal de exploración.

## 8. Evaluación y demostración

La evaluación se centra en comprobar que cada requisito se traduce en una funcionalidad verificable. A nivel de datos, la caché incluida permite arrancar la aplicación con 105 destinos, reconstruir el grafo y generar el índice semántico sin depender de una extracción completa en cada ejecución.

| Indicador           | Evidencia en el proyecto                                                            | Valor observado        |
|---------------------|-------------------------------------------------------------------------------------|------------------------|
| Volumen de destinos | <code>data/destinos.json</code> : entidades turísticas normalizadas.                | 105 destinos           |
| Cobertura temática  | Ciudades/destinos, museos y patrimonio UNESCO.                                      | 54 / 30 / 21           |
| Grafo enlazado      | Recursos locales con <code>owl:sameAs</code> hacia Wikidata.                        | 105 enlaces            |
| Interoperabilidad   | Exportación <code>/api/ttl</code> , consola SPARQL y JSON REST.                     | TTL + SPARQL + REST    |
| Robustez            | Caché, timeouts, carga OSM en segundo plano, fallback TF-IDF y respuesta local RAG. | Degradación controlada |

Para una demostración de 10 minutos, el flujo más completo es: cargar caché o APIs, filtrar museos/UNESCO en el mapa, abrir un destino y consultar su clima, buscar “arte medieval y catedrales góticas”, preguntar al guía RAG por recomendaciones culturales, visualizar el grafo y ejecutar una consulta SPARQL de conteo por tipo. Así se cubren fuentes abiertas, preprocesado, modelado semántico, interacción, recomendación y accesibilidad.

## 9. Valoración técnica y conclusiones

El programa cumple los requisitos del trabajo: integra más de dos fuentes abiertas, define un pipeline de limpieza y fusión, propone e implementa una arquitectura completa, expone más de cuatro funcionalidades basadas en datos integrados y presenta una interfaz navegable con mapa, grafo y consultas. También añade elementos valorados positivamente por la rúbrica: recomendación semántica, chatbot RAG, SHACL, accesibilidad y posibilidad de integración con otras aplicaciones mediante REST, Turtle y SPARQL.

Como limitaciones, la carga completa depende de conectividad y de la disponibilidad de APIs públicas; Overpass puede ser lento y por eso se ejecuta en segundo plano; el LLM requiere clave externa; y la caché local revisada todavía no contiene elementos OSM persistidos. Aun así, la arquitectura está preparada para degradar de forma controlada: caché si falla la red, TF-IDF si faltan embeddings y respuesta local si no hay LLM.

Como evolución natural, el sistema podría incorporar persistencia periódica de los POIs recuperados desde OSM, perfiles de usuario para ajustar recomendaciones, validaciones SHACL más estrictas por tipo de destino y despliegue público con actualización programada de datos. Estas mejoras no cambian el núcleo semántico del proyecto, sino que lo convierten en una base extensible para un servicio turístico reutilizable y mantenible.